> It might seem easier to simply access the customer emails whenever we need them rather than pluck out the data we want and store it in a database. There are two reasons not to do this.
>
> The first and most practical reason is about separation of concerns. Our support requests only need part of what is in the emails, but they also might need more metadata than the customer sends us. To keep our code organized and clean, it's better to store what we need explicitly.
>
> The second reason is one of Rails' famously held opinions. Rails arranges for all emails to be deleted after thirty days. The reasoning is that emails contain personal data that we don't want to hold onto unnecessarily.
>
> Protecting the personal data of your customers is a good practice, and it's one that's more and more required by law. For example, the European General Data Protection Regulation (GDPR) requires, among other things, that you delete any personal data you have within one month of a request to do so. By auto-deleting personal data every thirty days, you automatically comply with this requirement.[87]

Now, restart your server and navigate to the conductor at **http://localhost:3000/rails/conductor/action_mailbox/inbound_emails**. Click "Deliver new inbound email" and send another email (remember to send it to **support@example.com**).

Now, quit your server and start up the Rails console. This will allow us to check that a new **SupportRequest** was created (remember we have to format this to fit in the book, so your output will be on fewer, longer lines):

```
> bin/rails console
irb(main):001:0> SupportRequest.first
   (1.5ms)  SELECT sqlite_version(*)
  SupportRequest Load (0.1ms)
    SELECT "support_requests".* FROM "support_requests"
      ORDER BY "support_requests"."id" ASC LIMIT ?  [["LIMIT", 1]]
=> #<SupportRequest
       id: 1,
```